02 — regfile.v Explained: The Register File

Oct 6, 2026 · @shiv

1. Overview

`regfile.v` holds the eight 8-bit registers R0–R7. Two registers can be read and one written in the same clock cycle, and a bypass returns a value that is being written in that same cycle.

Item	Value
File	design_sources/regfile.v (65 lines)
Module name	regfile
Pipeline stage	Read in Stage 2 (ID); written in Stage 5 (WB)
Type of logic	Clocked write, combinational read
Storage	8 × 8 = 64 flip-flops
Calls (instantiates)	Nothing – leaf module
Called by	risc8_5stage_core.v, instance u_rf, lines 131–140
Also read by	tb_risc8_5stage.v (dut.u_rf.r[...]) to check results

1.1 Where it sits

```
tb_risc8_5stage.v
└── risc8_5stage_core.v
    ├── imem.v
    ├── control_unit.v
    ├── regfile.v        <== THIS FILE
    ├── hazard_unit.v
    ├── alu.v
    └── dmem.v
```

1.2 Ports

Port	Direction	Width	Meaning	Connected to (in risc8_5stage_core.v)
clk	input	1	Clock	clk
rst	input	1	Synchronous reset, active high	rst
ra_addr	input	3	Register to read on port A	id_ra_addr (rs1, or rd for ADDI)
ra_data	output	8	Value of that register	id_ra_data → ID/EX register
rb_addr	input	3	Register to read on port B	id_rb_addr (rs2, or rd for ST)
rb_data	output	8	Value of that register	id_rb_data → ID/EX register
we	input	1	Write enable	memwb_reg_we
wr_addr	input	3	Register to write	memwb_rd
wr_data	input	8	Value to write	wb_data (= memwb_data)

1.3 Dependencies

Modules, functions or tasks called inside this file: none. It is a leaf module, built only from Verilog operators.

Used by (file)	How	Instance	Lines
risc8_5stage_core.v	Module instance	u_rf	131–140
tb_risc8_5stage.v	Reads array r inside task check_reg	dut.u_rf	96–100

1.4 Verilog constructs used

Construct	Meaning in plain words
`reg [7:0] r [0:7]`	Array of 8 registers, 8 bits each

Construct	Meaning in plain words
`always @(posedge clk)`	Clocked block: builds flip-flops that update on the rising edge
`<=`	Non-blocking assignment: all flip-flops update together at the edge
`for` loop	Repeats a statement; synthesis unrolls it into parallel hardware
`cond ? x : y`	Conditional operator, which becomes a 2-to-1 multiplexer
`assign`	Continuous combinational connection

2. Code walkthrough, part by part

2.1 File header (lines 1–24)

The comment block at the top explains the two read ports, the write port and the write-through bypass (lines 11–17). It also records the dependencies: risc8_5stage_core.v creates this module as `u_rf`, and tb_risc8_5stage.v looks inside it to check results. It calls nothing.

2.2 Timescale and ports (lines 26–41)

```
`timescale 1ns/1ps

module regfile (
    input  wire       clk,      // System clock
    input  wire       rst,      // Synchronous reset, active high
    // ---- Read port A (Stage 2, ID) ----
    input  wire [2:0] ra_addr,
    output wire [7:0] ra_data,
    // ---- Read port B (Stage 2, ID) ----
    input  wire [2:0] rb_addr,
    output wire [7:0] rb_data,
    // ---- Write port (Stage 5, WB) ----
    input  wire       we,       // write enable
    input  wire [2:0] wr_addr,  // destination register
    input  wire [7:0] wr_data   // value to write
);
```

- `` `timescale 1ns/1ps `` tells the simulator that a delay of 1 means 1 ns. It does not affect the hardware.
- Each **read port** takes a 3-bit register number (0–7) and returns that register's 8-bit value.
- The **write port** takes a write enable `we`, a register number `wr_addr` and the value `wr_data`.
- `clk` and `rst` are needed because writing happens on the clock edge.

2.3 The storage array (lines 43–45)

```
    // 8 registers x 8 bits: r[0] = R0 ... r[7] = R7
    reg [7:0] r [0:7];
    integer i;
```

`reg [7:0] r [0:7]` declares an **array** of 8 elements, each 8 bits: `r[0]` is R0 and `r[7]` is R7. Because they are written in a clocked block, they become 64 flip-flops. `integer i` is only a loop counter for the reset loop and does not become hardware.

2.4 Writing on the clock edge (lines 47–55)

```
    always @(posedge clk) begin
        if (rst)
            for (i = 0; i < 8; i = i + 1) r[i] <= 8'h00;   // clear all
        else if (we)
            r[wr_addr] <= wr_data;
    end
```

- `always @(posedge clk)` runs once at every rising clock edge, so everything inside becomes flip-flops.
- On **reset**, the `for` loop clears all 8 registers. Synthesis unrolls the loop into 8 parallel clears.
- Otherwise, if `we` = 1, the register chosen by `wr_addr` receives `wr_data`.
- `<=` is a **non-blocking** assignment, the correct choice for flip-flops: all registers update together at the edge.

Link to another file: in risc8_5stage_core.v, `we`, `wr_addr` and `wr_data` come from the MEM/WB pipeline register (`memwb_reg_we`, `memwb_rd`, `wb_data`). This is Stage 5, write-back.

2.5 Reading, with the bypass (lines 57–63)

```
    assign ra_data = (we && (wr_addr == ra_addr)) ? wr_data : r[ra_addr];
    assign rb_data = (we && (wr_addr == rb_addr)) ? wr_data : r[rb_addr];
```

Each read port is a `?:` expression, which becomes a multiplexer:

1. **If** a write is happening this cycle (`we`) **and** it is to the register being read (`wr_addr == ra_addr`), output the value being written (`wr_data`).
2. **Otherwise** output the stored value `r[ra_addr]`.

Reading needs no clock, so the value is ready in the same cycle as the address. Without the bypass, an instruction in ID would read the old value, because the new value is only stored at the end of the cycle.

2.6 End of module (line 65)

`endmodule` closes the module.

3. Bypass example, mistakes and viva

3.1 Bypass example from the test program

XOR R5 is at address 0x04, and SUB R7, R7, R5 is at 0x07, three instructions later. When SUB is in ID, XOR is in WB, writing R5 in that same cycle.

What happens in that cycle	Without bypass	With bypass
WB writes R5 = 0x33, while ID reads R5 for SUB	SUB reads the old R5 = 0x00 (wrong)	SUB reads 0x33 (correct)

3.2 Common mistakes

- Making the read clocked (registered) adds a cycle of delay and breaks the pipeline timing.
- Leaving out the bypass: instructions three apart would read stale values.
- Using blocking `=` for the register writes: simulation may not match the hardware.

3.3 Viva questions

Q1. How many registers, and how wide? Eight registers (R0–R7), each 8 bits.

Q2. Why two read ports? Instructions such as ADD R3, R1, R2 need two operands in the same cycle.

Q3. What is the bypass for? When WB writes a register in the same cycle that ID reads it, the bypass returns the new value, so no stall is needed.

Q4. Is reading clocked? No. Reading is combinational, so the value is ready in the same cycle as the address.

Q5. Is R0 always zero, as in MIPS or RISC-V? No. In this design R0 is an ordinary register.